

Python Built-in Functions

`range()` and `enumerate()`

What is range()?

range() generates a sequence of integers

It is LAZY – returns a range object, not a list

Memory efficient: stores only start, stop, step

Three syntax variants:

```
range(stop)           # 0 to stop-1
range(start, stop)    # start to stop-1
range(start, stop, step) # with step size
```

```
# Convert to list to see values
list(range(5))          # [0, 1, 2, 3, 4]
```

Output:

```
[0, 1, 2, 3, 4]
```

```
Type: <class 'range'>
```

range() — Three Syntax Variants



Variant 1: range(stop)

Starts at 0, ends before stop

Variant 2: range(start, stop)

Starts at start, ends before stop

Variant 3: range(start, stop, step)

Adds step increment (can be negative)

```
# Variant 1
for i in range(5):
    print(i, end=' ')
```

```
# Variant 2
for i in range(2, 7):
    print(i, end=' ')
```

```
# Variant 3
for i in range(10, 0, -2):
    print(i, end=' ')
```

Output:

```
0 1 2 3 4
2 3 4 5 6
10 8 6 4 2
```

range() in for Loops

Most common use: iterate N times

Access list elements by index

Generate sequences for algorithms

Avoid manual index management

```
# Repeat action N times
for i in range(3):
    print(f"Iteration {i}")
```

```
# Access by index
fruits = ['apple', 'banana', 'cherry']
for i in range(len(fruits)):
    print(f"{i}: {fruits[i]}")
```

Output:

```
Iteration 0
Iteration 1
Iteration 2
```

```
0: apple
1: banana
2: cherry
```

range() — Common Pitfalls

`range(5)` gives 0-4, NOT 0-5!

`range` is NOT a list — must convert

Floats are NOT allowed in `range()`

Negative step requires `start > stop`

Output:

```
TypeError: 'float' object  
cannot be interpreted as integer
```

```
[0.  0.1 0.2 ... 0.9]
```

```
[]
```

```
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
```

```
# WRONG: range with float  
range(0, 1, 0.1) # TypeError!
```

```
# CORRECT: use numpy or manual  
import numpy as np  
np.arange(0, 1, 0.1)
```

```
# WRONG: negative step with start < stop  
range(0, 10, -1) # Empty!
```

```
# CORRECT:  
range(10, 0, -1) # 10, 9, ..., 1
```

What is enumerate()?



enumerate() adds a counter to any iterable

Returns (**index**, **value**) pairs as tuples

Eliminates manual index tracking

Cleaner, more Pythonic code

```
# Without enumerate (old way)
```

```
i = 0
```

```
for item in items:
```

```
    print(i, item)
```

```
    i += 1
```

```
# With enumerate (Pythonic)
```

```
for i, item in enumerate(items):
```

```
    print(i, item)
```

Output:

Both produce:

0 apple

1 banana

2 cherry

enumerate() — Syntax & Options



Syntax: `enumerate(iterable, start=0)`

`start` parameter lets you begin at any number

Default start is 0

Works with any iterable: list, tuple, string, dict

enumerate() — Syntax & Options

```
colors = ['red', 'green', 'blue']

# Default start (0)
for i, c in enumerate(colors):
    print(i, c)

# Custom start (1)
for i, c in enumerate(colors, 1):
    print(i, c)

# With string
for i, ch in enumerate("ABC", 1):
    print(f"Letter {i}: {ch}")
```

Output:

```
0 red
1 green
2 blue

1 red
2 green
3 blue

Letter 1: A
Letter 2: B
Letter 3: C
```

range() vs enumerate() — When to Use?



Use range() when:

- You need numbers 0 to N-1
- Repeating an action N times
- Generating numeric sequences
- Index-based iteration (less Pythonic)

Use enumerate() when:

- You need both index AND value
- Iterating over a collection
- Creating numbered lists
- More readable, Pythonic code

```
# range() – numeric sequence
squares = [x**2 for x in range(5)]
# [0, 1, 4, 9, 16]

# enumerate() – index + value
for i, name in enumerate(['A','B','C']):
    print(f"{i+1}. {name}")

# 1. A
# 2. B
# 3. C
```